FSD-2 External Lab — Daily Expense Tracker

AIM

To develop a full-stack **Daily Expense Tracker** web application using **React.js** for the frontend and **Node.js with Express.js** for the backend. The user can add, view, edit and delete daily expenses and see the total amount spent. The project is managed using **Git and GitHub**.

PROCEDURE

1. Install Node.js, Git and VS Code.
2. Create a project folder, open it in VS Code and run `npm init -y`.
3. Install the packages:
   `npm install express cors react react-dom`
   `npm install -D vite @vitejs/plugin-react concurrently`
4. Create `server.js` (backend) using Express. Use the `cors()` and `express.json()` middleware.
5. Write REST APIs in server.js:
   - `GET /api/expenses` → view all expenses
   - `POST /api/expenses` → add an expense
   - `PUT /api/expenses/:id` → update an expense
   - `DELETE /api/expenses/:id` → delete an expense

   The data is stored in the `data/expenses.json` file.
6. Create `src/App.jsx` (frontend). Use `useState` to store the expenses and `useEffect` + `fetch()` to load them from the backend.
7. Create the components `Navbar`, `Summary`, `ExpenseForm` and `ExpenseList`, and pass data and functions to them using props.
8. In `vite.config.js`, set a proxy so that `/api` requests go to `http://localhost:5000`.
9. Push the code to GitHub:

```
git init
git add .
git commit -m "Daily Expense Tracker"
git remote add origin <repository-url>
git push -u origin main
```

10. In the lab, clone and run:

```
git clone <repository-url>
cd <project-folder>
npm install
npm run dev
```

11. Open `http://localhost:5173` in the browser and test adding, editing and deleting expenses.

MAIN CODE — FRONTEND (src/App.jsx)

```
import { useEffect, useState } from "react";
import Navbar from "./components/Navbar";
import Summary from "./components/Summary";
import ExpenseForm from "./components/ExpenseForm";
import ExpenseList from "./components/ExpenseList";

function App() {
  const [expenses, setExpenses] = useState([]);
  const [editItem, setEditItem] = useState(null);

  const loadExpenses = () => {
    fetch("/api/expenses")
      .then((res) => res.json())
      .then((data) => setExpenses(data));
  };

  useEffect(() => {
    loadExpenses();
  }, []);

  const deleteExpense = async (id) => {
    if (!window.confirm("Delete this expense?")) return;
    await fetch(`/api/expenses/${id}`, { method: "DELETE" });
    loadExpenses();
  };

  const total = expenses.reduce((sum, e) => sum + Number(e.amount), 0);

  return (
    <>
      <Navbar />
      <div className="container">
        <Summary expenses={expenses} total={total} />
        <ExpenseForm editItem={editItem} setEditItem={setEditItem} onSave=
{loadExpenses} />
        <ExpenseList expenses={expenses} onEdit={setEditItem} onDelete=
{deleteExpense} />
      </div>
    </>
  );
}

export default App;
```

MAIN CODE — BACKEND (server.js)

```
const express = require("express");
const cors = require("cors");
const { readData, saveData } = require("./utils/db");
const validate = require("./utils/validate");
const budgetRoutes = require("./routes/budgetRoutes");

const app = express();
app.use(cors());
app.use(express.json());

app.get("/api/expenses", (req, res) => res.json(readData()));

app.post("/api/expenses", validate, (req, res) => {
  const expense = { id: Date.now().toString(), ...req.body };
  saveData([...readData(), expense]);
  res.status(201).json(expense);
});

app.put("/api/expenses/:id", validate, (req, res) => {
  const expenses = readData().map((e) => (e.id === req.params.id ? { ...e,
...req.body } : e));
  saveData(expenses);
  res.json({ message: "Expense updated" });
});

app.delete("/api/expenses/:id", (req, res) => {
  saveData(readData().filter((e) => e.id !== req.params.id));
  res.json({ message: "Expense deleted" });
});

app.use("/api/budget", budgetRoutes);

app.listen(5000, () => console.log("Server running on port 5000"));
```

EXPECTED OUTPUT

1. The terminal shows `Server running on port 5000` and `VITE ready – Local: http://localhost:5173`.
2. The browser opens the Daily Expense Tracker page. It has a navbar and four cards: Total Expenses, This Month, Today and Highest.
3. Enter an expense (for example: Lunch, ₹120, Food, today's date) and click **Add Expense**. A message says "Expense added", the expense appears in the table, and all the totals update.
4. Click **Edit**. The form fills with that expense. Change the amount and click **Update Expense**, and the table updates.
5. Click **Delete** and confirm. The expense is removed and the total decreases.

6. Below the table, the Monthly Budget progress bar, the Category-wise bars and the Last 7 Days chart show the spending.
7. Search, the category/month filters and sorting all work, and **Export CSV** downloads the list.
8. If the description is empty or the amount is 0, an error message is shown.
9. The data stays saved after the server restarts, because it is stored in `data/expenses.json`.

ORIGINAL OUTPUT

(Write what you actually saw, for example:) The application ran successfully on http://localhost:5173. Expenses were added, edited and deleted successfully. The total, monthly and today's amounts were updated correctly, and the data was stored in the JSON file.

HOW TO REMEMBER

server.js — 6 blocks (top to bottom)

#	Block	Key line
1	5 requires	express, cors, `{ readData, saveData }`, validate, budgetRoutes
2	app + 2 middlewares	`app.use(cors())`, `app.use(express.json())`
3	**GET** (1 line)	`res.json(readData())`
4	**POST** (3 lines)	make expense with `id: Date.now()` → save `[...old, new]` → 201
5	**PUT** (3 lines)	`map`: if id matches, merge `{ ...e, ...req.body }` → save → "updated"
6	**DELETE** (2 lines), budget route, listen	`filter` out the id → save; `app.listen(5000)`

Trick: **G-P-P-D** = Get, Post, Put, Delete.

- **POST and PUT have `validate`** between the URL and `(req, res)`. It is a middleware that checks the data (400 error) and, for PUT, whether the id exists (404 error).
- **POST adds** → spread `[...readData(), expense]`
- **PUT changes** → `map`
- **DELETE removes** → `filter`

Viva answer: "*`validate` is a middleware. It runs before the route, checks the input and calls `next()` if the data is valid.*"

App.jsx — 5 blocks

#	Block	Key line
1	imports	2 hooks + 4 components (Navbar, Summary, ExpenseForm, ExpenseList)
2	2 states	`expenses` = `[]`, `editItem` = `null`
3	loadExpenses + useEffect	fetch → res.json() → setExpenses; useEffect with `[]`
4	deleteExpense + total	confirm → fetch DELETE → loadExpenses; `reduce` for total
5	return	Navbar, then container with Summary, ExpenseForm, ExpenseList

Props trick: **Summary** gets *expenses + total*. **Form** gets *editItem, setEditItem, onSave*. **List** gets *expenses, onEdit, onDelete*.

If something goes wrong in the lab

Problem	Fix
`npm install` fails (no internet)	Ignore it and run `npm run dev`. `node_modules` is already in the repo.
`EADDRINUSE: port 5000`	Close the old terminal (Ctrl+C) or restart VS Code, then run again
Browser didn't open	Open http://localhost:5173 yourself
Page shows but no data	Check that the terminal shows `[server] Server running on port 5000`

EXAM DAY — STEP BY STEP

Before the exam: read these notes once, and write server.js and App.jsx once from memory.

Step 1: write the paper first. Aim → Procedure → App.jsx → server.js → Expected output. Leave Original output empty for now.

Step 2: check the PC. In VS Code, open the terminal (menu Terminal → New Terminal) and run `node -v` and `git -v`. Both must show a version number.

Step 3: clone and run

```
cd Desktop
git clone
https://github.com/somanagakiran/Daily_Expense_Tracker.git
cd Daily_Expense_Tracker
npm install
npm run dev
```

Wait for **both** lines: `[server] Server running on port 5000` and `Local: http://localhost:5173/`.

Step 4: show the examiner, in this order

1. **Add:** Lunch, ₹100, Food → Add Expense → the total increases
2. **Edit:** change it to ₹150 → Update → the change shows
3. **Delete:** click Delete and confirm → it is removed and the total decreases
4. **Validation:** click Add with an empty description → an error message shows
5. **Extras:** budget bar, category bars, 7-day chart, search, filter, Export CSV, dark mode
6. **Data is saved:** Ctrl + C → `npm run dev` again → refresh → the data is still there

Step 5: write the Original output.

VIVA QUESTIONS & ANSWERS

About the project

Q: Why did you choose this project?

> Many people spend money every day on food, travel and bills, but they don't track it, so at the end of the month they don't know where the money went. My app records daily expenses and shows the spending clearly, so the user can control their budget. It is also a good full-stack project because it uses all four CRUD operations.

Q: What is unique in your project?

1. Edit/update of expenses, not just add and delete
2. A monthly budget with a progress bar that turns green, orange or red
3. Category-wise spending and a last 7 days chart, made without any chart library
4. Search, filter by category and month, and sorting
5. Export to CSV, which opens in Excel
6. Validation on both the frontend and the backend
7. Dark mode and a mobile-friendly design
8. Data stays saved in a JSON file, so no database setup is needed

Q: What technologies did you use?

React with Vite for the frontend, Node.js with Express for the backend, a JSON file for storage, and Git and GitHub for version control.

Backend (server.js)

Q: What is GET?

It is used to **read** data. `GET /api/expenses` returns all expenses.

Q: What is POST?

It is used to **add** new data. `POST /api/expenses` adds an expense.

Q: What is PUT?

It is used to **update** data. `PUT /api/expenses/:id` updates the expense with that id.

Q: What is DELETE?

It is used to **remove** data. `DELETE /api/expenses/:id` deletes that expense.

Q: What is CRUD?

Create, Read, Update, Delete, which are POST, GET, PUT and DELETE.

Q: What is Express?

> A Node.js framework that makes it easy to create a server and APIs.

Q: What is middleware?

> A function that runs **before** the route handler. `express.json()` reads the JSON body, `cors()` allows requests from the frontend, and `validate` checks the input and calls `next()` if the data is valid.

Q: Why use cors?

> The frontend (5173) and backend (5000) run on different ports. The browser blocks these requests by default, and CORS allows them.

Q: What is express.json()?

> It converts the JSON request body into a JavaScript object, so we can use `req.body`.

Q: What are req and res?

> `req` is the request coming from the client. `res` is the response we send back.

Q: What is req.params.id?

> The id taken from the URL. For example, in `/api/expenses/123`, the id is 123.

Q: What are status codes?

> 200 = OK, 201 = Created, 400 = Bad request (invalid data), 404 = Not found, 500 = Server error.

Q: Where is the data stored?

> In `data/expenses.json`. `readData()` reads the file and `saveData()` writes it, using Node's `fs` module.

Q: Why not MongoDB?

> For a small personal tracker, a JSON file is enough and needs no installation. It can be moved to MongoDB later.

Q: How is the id created?

With `Date.now()`, the current time in milliseconds, so every id is unique.

Q: What are map, filter and spread?

Spread `[...old, new]` adds an item (POST). `map` changes an item (PUT). `filter` removes an item (DELETE).

Frontend (App.jsx)

Q: What is React?

A JavaScript library for building user interfaces using components.

Q: What is a component?

A reusable piece of the UI. Mine are Navbar, Summary, ExpenseForm and ExpenseList.

Q: What is useState?

A hook that stores data that can change. When it changes, React updates the screen. I use it for `expenses` and `editItem`.

Q: What is useEffect?

A hook that runs code after the component loads. I use it to fetch expenses when the page opens. The `[]` means it runs only once.

Q: What are props?

They pass data from a parent component to a child. For example, `expenses` is passed to ExpenseList.

Q: What is fetch?

A browser function used to call APIs. I use it to call my backend.

Q: What is async/await?

It waits for a task, like an API call, to finish before going to the next line.

Q: What is reduce?

> It combines all the array items into one value. I use it to calculate the total amount.

Q: What is JSX?

> HTML-like syntax written inside JavaScript in React.

Q: What is Vite?

> A fast development server and build tool for React. It runs the frontend on port 5173.

Q: What is the proxy in vite.config.js?

> It sends all `/api` requests from the frontend to the backend at `localhost:5000`.

Git & npm

Q: What are Git and GitHub?

> Git is a version control system that tracks changes in code. GitHub is a website that stores Git repositories online.

Q: Explain the git commands.

- `git init` : create a new repository
- `git add .` : add all files to staging
- `git commit -m "msg"` : save a snapshot with a message
- `git remote add origin <url>` : link to GitHub
- `git push` : upload to GitHub
- `git clone <url>` : download the repo to a new PC

Q: What does npm install do?

> It installs all the packages listed in `package.json`.

Q: What does npm run dev do?

> It starts both the backend and the frontend together using `concurrently`.

Viva tips

- Give one- or two-line simple answers.
- If you don't know, say: "I'm not sure, sir, but I think..."
- Point to the screen while you explain.